Week 6

Rainbow DQN and Modern Advances (BBF, SimBa)

STUDY-RL — a self-paced Deep RL curriculum

November 8, 2026

Contents

Learning objectives 3

1 Motivation and intuition 4

2 Mathematical formulation 5
- 2.1 Double DQN . . . 5
- 2.2 Dueling DQN . . . 5
- 2.3 Prioritized Experience Replay . . . 5
- 2.4 Noisy Networks . . . 6
- 2.5 Distributional RL: C51, QR-DQN, IQN . . . 7
  - 2.5.1 C51 . . . 7
  - 2.5.2 QR-DQN . . . 7
  - 2.5.3 IQN . . . 7
- 2.6 Rainbow . . . 8
- 2.7 BBF (Schwarzer 2023): scaling a value-based agent . . . 8

3 Algorithms 9

4 Implementation walkthrough 10

5 Empirical results 12
- 5.1 Statistics bridge: A return distribution is not an uncertainty interval for its mean . 12

6 Exercises 15

7 Further reading 16

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Derive the Double DQN target and explain how it reduces coupled selection/evaluation error in vanilla DQN.
2. Derive the Dueling architecture's mean-subtraction $Q = V + (A - \bar{A})$ and explain the differences between mean-subtraction and max-subtraction.
3. Derive the Prioritized Experience Replay importance-sampling weight $w_i = (NP(i))^{-\beta}$ from first principles and explain the α, β annealing schedule.
4. Implement Noisy Networks for parameter-space exploration and replace the ε-schedule.
5. Summarise C51, QR-DQN, and IQN, the three pillars of distributional RL.
6. State the Rainbow combination and the BBF (Schwarzer 2023) recipe: high replay ratio, shrink-and-perturb resets, EMA target, n-step + γ annealing.
7. Connect the importance-weighted PER objective to the OPE counterfactual reweighting used in synthetic insurance / ATS matching.

1 Motivation and intuition

Week 5 left us with a working DQN that solves LunarLander after a few hundred thousand steps. Rainbow (Hessel et al., 2018) showed that combining six small enhancements — Double, Dueling, PER, n-step, NoisyNets, and a distributional value head — produces an agent that comprehensively beats vanilla DQN on Atari. The contribution of each component depends on the game, training budget, and the other components present. Ablation is needed to establish whether a particular combination improves a specified metric.

Intuition. Vanilla DQN is a one-trick pony: minimise the squared Bellman residual under a frozen target. Rainbow asks "what *else* could the same data tell us?" — and answers it in six different ways:

- Use the data to debias the max (Double).
- Decouple "how good is this state" from "which action is best" (Dueling).
- Sample the data in proportion to how surprised we are (PER).
- Look beyond one step into the future (n-step).
- Make exploration a property of the network, not the controller (NoisyNets).
- Model the full return *distribution*, not just its mean (C51/QR-DQN/IQN).

BBF (Schwarzer 2023) is the modern descendant: a coordinated recipe involving replay, representation learning, resets, schedules and larger networks. It is not simply a verbatim Rainbow implementation with two extra switches. It achieved strong Atari-100k results in its 2023 evaluation; this is a historical result, not a claim about the current leaderboard. SimBa (Lee 2024) is the architectural analogue for continuous control. Both are studied in detail in Week 10.

2 Mathematical formulation

2.1 Double DQN

Proposition 2.1.1 (Maximisation bias)**.** *If each action estimate is unbiased, Jensen's inequality gives* $\mathbb{E}[\max_a \hat{Q}_a] \geq \max_a \mathbb{E}[\hat{Q}_a]$. *For* $K \geq 2$ *mean-zero* σ*-sub-Gaussian errors,* $\mathbb{E}[\max_a \epsilon_a] \leq \sigma\sqrt{2\log K}$*; thus this also bounds the overestimation gap. Strict inequality need not hold.*

Proof. For $t > 0$, $\mathbb{E}\max_a \epsilon_a \leq \log K/t + t\sigma^2/2$ by log-sum-exp, Jensen and the moment-generating-function bound. Minimise in t. □

Double DQN (van Hasselt 2016) decouples selection from evaluation in the target:

$$y_i^{\text{DDQN}} = r_i + \gamma(1 - d_i)\, Q_{\theta^-}\Big(s_i',\ \arg\max_{a'} Q_\theta(s_i', a')\Big). \tag{2.1}$$

The argmax uses the online net (with all its noise) but the value is read off the target net. The networks share training history and are not independent; this reduces selection/evaluation coupling but need not eliminate all bias. It requires an additional next-state online-network evaluation.

2.2 Dueling DQN

Decompose

$$Q(s,a) = V(s) + A(s,a) - \bar{A}(s),$$

Here A denotes the raw advantage-head output and $\bar{A}$ its action mean. The centred residual $A - \bar{A}$ sums to zero, so $V(s)$ equals the uniform mean of $Q(s,\cdot)$. This need not equal the value under the behaviour policy. Adding a constant to every raw advantage still leaves Q unchanged: the raw head itself is not uniquely identifiable. Mean subtraction fixes the effective value/residual decomposition. Max subtraction instead makes $V(s) = \max_a Q(s,a)$ and introduces an argmax-dependent gradient; both aggregations are considered in the original dueling paper.

2.3 Prioritized Experience Replay

In uniform replay, every transition is sampled with probability $1/|\mathcal{D}|$. PER samples in proportion to a *priority* p_i:

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}, \quad p_i = |\delta_i| + \epsilon,$$

where δ_i is the most recent TD error for transition i, $\alpha \in [0,1]$ controls how aggressive the prioritisation is ($\alpha = 0$ recovers uniform), and ϵ keeps zero-error transitions sampleable. Newly-added transitions get the maximum existing priority to encourage prompt sampling; this does not guarantee that every transition is sampled before eviction.

Theorem 2.3.1 (IS-weight correction)**.** *Let* $f(\theta)$ *be the expected DQN loss under uniform sampling, and* $\hat{f}(\theta)$ *the empirical loss under prioritised sampling. Without correction, the stochastic gradient is biased. The* ***importance-sampling weight***

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta = (NP(i))^{-\beta}$$

recovers an unbiased gradient when $\beta = 1$ and trades bias for variance for $\beta < 1$. Normalising $w_i \to w_i / \max_j w_j$ bounds the update magnitude ($w_i \in [0, 1]$), but changes the exact identity: a fixed full-buffer maximum rescales the expected gradient, while a sampled minibatch maximum can additionally bias it. Annealing $\beta : 0.4 \to 1.0$ over training (Schaul 2016) interpolates between low-variance (biased) and full-correction.

Proof. Standard importance-sampling identity: $\mathbb{E}_P[wX] = \mathbb{E}_U[X]$ when $w = U/P$ pointwise. Power β tempers the weight to manage variance. □

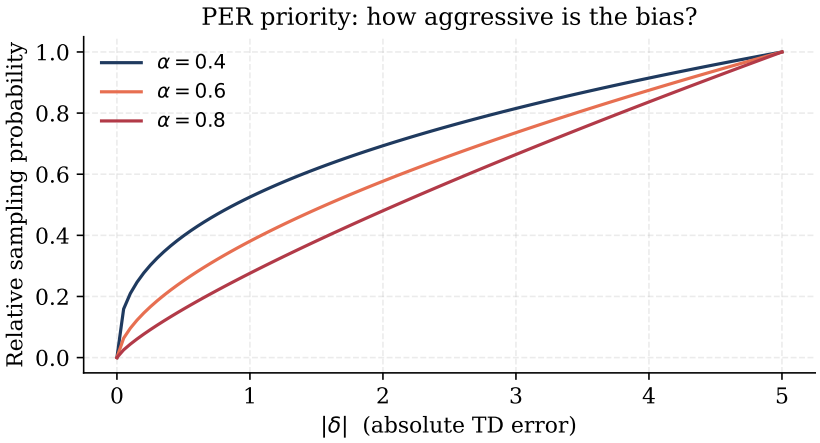


Figure 2.1: Effect of the prioritisation exponent α. $\alpha = 0$ is uniform; larger α concentrates sampling on high-TD-error transitions.

Sumtree implementation. The standard data structure is a *sumtree* with $O(\log N)$ per push/sample/update. Our minimal implementation in `common/replay.py` is $O(N)$ per sample (acceptable for buffers $\leq 10^5$, which covers all course exercises).

2.4 Noisy Networks

Factorised Gaussian noise. Replace each `Linear` layer's affine map $y = Wx + b$ with

$$y = (\mu^W + \sigma^W \odot \epsilon^W)x + (\mu^b + \sigma^b \odot \epsilon^b),$$

where the noise is *factorised* as $\epsilon^W_{ij} = f(\epsilon^{\text{out}}_i) \cdot f(\epsilon^{\text{in}}_j)$ with $f(x) = \text{sign}(x)\sqrt{|x|}$. This uses only $O(\dim_{\text{in}} + \dim_{\text{out}})$ fresh Gaussian draws per sample (versus $O(\dim_{\text{in}} \cdot \dim_{\text{out}})$ for independent). The σ parameters are *learned* alongside the μ; they can increase or decrease and are not calibrated posterior uncertainty estimates. Shared input/output factors make the resulting weight noises dependent.

Result: exploration is now parameter-space, state-dependent, and self-tuning. Drops ε from the hyper-parameter list.

Common pitfall

At evaluation time set the network to `eval` mode so noise is disabled. The distinction is invisible to PyTorch's autograd, but the *policy* you ship is the μ-only deterministic one.

2.5 Distributional RL: C51, QR-DQN, IQN

2.5.1 C51

Model the return distribution $Z(s,a)$ as a discrete distribution on a fixed support $\{z_i\}_{i=0}^{N-1}$ with $N = 51$ atoms, e.g. $z_i = V_{\min} + i\Delta z$, $\Delta z = (V_{\max} - V_{\min})/(N-1)$. The distributional Bellman operator is

$$\mathcal{T}^\pi Z(s,a) \overset{D}{=} R(s,a) + \gamma Z(S', A').$$

After applying $\mathcal{T}$, the support shifts to $\{r + \gamma z_j\}$ which is generally *not* on $\{z_i\}$, so we project. The *categorical projection* Φ scatters each transformed atom $\tilde{z}_j = \text{clip}(r + \gamma z_j, V_{\min}, V_{\max})$ onto its two neighbouring grid atoms with interpolation weights:

$$(\Phi \mathcal{T} Z)_i = \sum_j p_j \left[1 - |\tilde{z}_j - z_i|/\Delta z\right]_+.$$

Train with cross-entropy between $\Phi\mathcal{T}Z$ and $Z_\theta(s,a)$:

$$\mathcal{L}_{\text{C51}} = -\sum_i (\Phi \mathcal{T} Z)_i \log Z_\theta(s,a)_i.$$

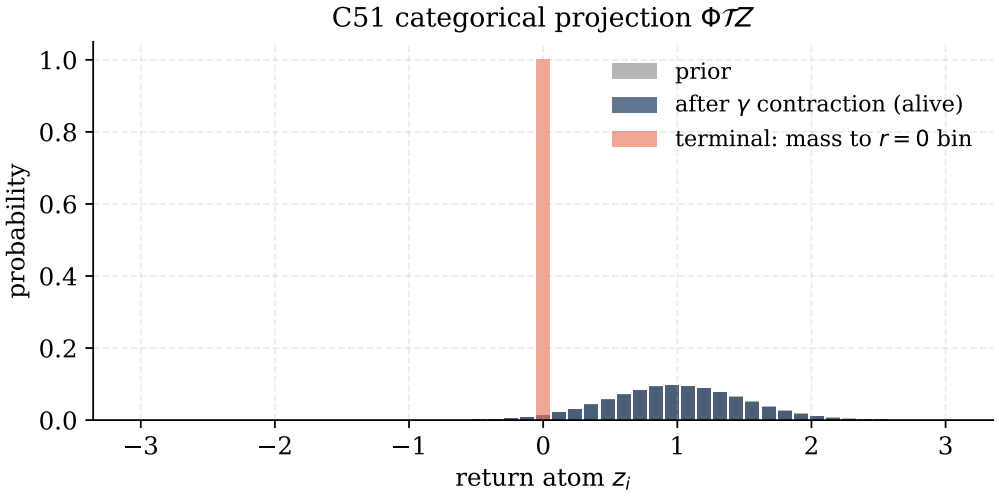


Figure 2.2: C51 categorical projection. A peak around $z = 1$ is contracted by $\gamma = 0.99$ when alive, and collapses onto the $r = 0$ bin when terminal.

2.5.2 QR-DQN

Drop the fixed support. Learn N *quantile locations* $\theta_i(s,a)$ at fixed fractions $\tau_i = (2i-1)/(2N)$. Loss is the asymmetric Huber quantile regression

$$\mathcal{L} = \sum_i \mathbb{E}_j \left[\rho_{\tau_i}^\kappa (y_j - \theta_i(s,a))\right], \qquad \rho_\tau^\kappa(u) = |\tau - \mathbb{1}(u < 0)|\, \mathcal{L}_\kappa(u)/\kappa,$$

where $\mathcal{L}_\kappa$ is the Huber loss of width κ. Wasserstein-friendly; no projection. Standard: $N = 200$, $\kappa = 1$.

2.5.3 IQN

Make the quantile network *implicit*: sample $\tau \sim U[0,1]$, condition the network on a cosine embedding $\phi(\tau) = \cos(i\pi\tau)$, output $Z_\tau(s,a)$. Variable sample efficiency, and distortion-risk

policies (CVaR, Wang's distortion) come for free by choosing how you sample τ at action time. Compare IQN and QR-DQN under the published evaluation protocol; there is no task-independent performance or compute ratio.

2.6 Rainbow

The Rainbow combination (Hessel 2018):

$$\text{Double} + \text{Dueling} + \text{PER} + \text{n-step } (n = 3) + \text{NoisyNets} + \text{C51}.$$

Ablation lesson: evaluate components under a fixed budget and protocol. Their interactions mean that effects are not independent or additive. The original Atari comparisons do not establish a universal percentage loss for removing any component.

2.7 BBF (Schwarzer 2023): scaling a value-based agent

BBF combines larger networks with replay, self-predictive representation learning, resets, and scheduled discount / multi-step targets. Its Atari-100k results depend on the complete recipe. The local shrink-and-perturb exercise $\theta \leftarrow \alpha\theta + (1 - \alpha)\theta_{\text{init}}$ illustrates one mechanism; it is not a full reproduction or a claim that these coefficients equal every parameter group's published reset schedule. Source and configuration: https://arxiv.org/abs/2305.19452.

Common pitfall

High replay ratios can magnify overfitting and loss of plasticity. Their interaction with resets, representation learning, and architecture is empirical; there is no universal replay-ratio threshold or requirement that every useful reset mechanism use LayerNorm and residual connections.

3 Algorithms

Algorithm 1: Rainbow-lite (Double + Dueling + PER + Noisy)

Input: Rainbow flags, PER α, β, target Polyak τ
Initialise Q_θ (with Noisy + Dueling), copy θ^-;
for *env step* $t = 1, 2, \ldots$ **do**
  Sample noise: reset_noise(Q_θ);
  $a_t \leftarrow \arg\max_a Q_\theta(s_t, a)$;
  Step env; push (s, a, r, s', d) into PER buffer;
  if $|\mathcal{D}| \geq$ *warmup* **then**
    Sample batch with weights $w_i = (NP(i))^{-\beta} / \max$;
    Compute Double-DQN target: $y_i = r_i + \gamma(1 - d_i)Q_{\theta^-}(s'_i, \arg\max_a Q_\theta(s'_i, a))$;
    Compute per-sample Huber loss; weight by w_i; mean-reduce;
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$;
    Update priorities: $p_i = |\delta_i| + \epsilon$;
    Polyak: $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$;

Complexity. Per step: one Noisy-noise reset ($O(\text{dim})$), one Double-DQN forward on online + target ($2\times$ vanilla DQN forward cost), batch backward. The additional passes and sampling work depend on the enabled components; measure wall-clock and learning curves rather than assume fixed overhead or gains.

4 Implementation walkthrough

The Rainbow-lite agent lives in `project/agent.py`; the ablation harness in `project/train.py`. The critical fragment is the learn step:

```
def learn(self):
    if len(self.buf) < max(self.cfg.batch_size, self.cfg.warmup_steps):
        self.step_count += 1
        return None
    if self.cfg.per:
        s, a, r, s2, d, idx, w = self.buf.sample(self.cfg.batch_size)
        w_t = torch.as_tensor(w, device=device)
    else:
        s, a, r, s2, d = self.buf.sample(self.cfg.batch_size)
        idx = None; w_t = torch.ones(self.cfg.batch_size, device=device)
    if self.cfg.noisy:
        self.q.reset_noise(); self.q_target.reset_noise()
    q_sa = self.q(s).gather(1, a.unsqueeze(1)).squeeze(1)
    with torch.no_grad():
        if self.cfg.double:
            a_star = self.q(s2).argmax(dim=1, keepdim=True)
            q_next = self.q_target(s2).gather(1, a_star).squeeze(1)
        else:
            q_next = self.q_target(s2).max(dim=1).values
        target = r + (1.0 - d) * self.cfg.gamma * q_next
    diff = q_sa - target
    l = torch.where(diff.abs() < 1.0, 0.5 * diff ** 2, diff.abs() - 0.5)
    loss = (w_t * l).mean()
    self.opt.zero_grad(); loss.backward()
    torch.nn.utils.clip_grad_norm_(self.q.parameters(), self.cfg.grad_clip)
    self.opt.step()
    if self.cfg.per:
        self.buf.update_priorities(idx, (target - q_sa).detach().cpu().numpy())
    self._soft_update()
```

The NoisyLinear layer is short enough to reproduce in full:

```
class NoisyLinear(nn.Module):
    @staticmethod
    def _f(x):
        return x.sign() * x.abs().sqrt()

    def reset_noise(self):
        self.eps_in = self._f(torch.randn(self.in_features))
        self.eps_out = self._f(torch.randn(self.out_features))

    def forward(self, x):
        if self.training:
            w_eps = self.eps_out.unsqueeze(1) * self.eps_in.unsqueeze(0)
            w = self.weight_mu + self.weight_sigma * w_eps
            b = self.bias_mu + self.bias_sigma * self.eps_out
```

```
        else:
            w, b = self.weight_mu, self.bias_mu
        return F.linear(x, w, b)
```

5 Empirical results

5.1 Statistics bridge: A return distribution is not an uncertainty interval for its mean

Example 5.1.1 (Original statistical teaching fixture)**.** A fixed policy has returns -1, 0 and 1 with probabilities 0.25, 0.5 and 0.25. Assume independent evaluation episodes for the mean-uncertainty calculation.

The return mean is 0 and variance is $0.25(1) + 0.5(0) + 0.25(1) = 0.5$. Its SD is $\sqrt{0.5} \approx 0.7071$. The sampling SD of the mean across 100 independent episodes is instead $\sqrt{0.5/100} \approx 0.07071$.

Distributional RL models variation of returns conditional on state and action. That outcome spread is different from epistemic uncertainty about learned probabilities or the mean. A wide categorical support is not a 95% confidence interval.

Check a changed assumption. Quadruple the independent evaluation episodes. What happens to the return distribution and the sampling SD of its mean?

Check. The modeled return distribution is unchanged. The sampling SD of the mean halves to approximately 0.03536 under the same independence and fixed-policy assumptions.

Statistical foundation. Siegrist § § 4.3, 6.2, PDF pp.460, 817. Section source: https://stats.libretexts.org/@go/page/10158. The course statistics companion records the supplied editions and one-based PDF pages. This fixture is not a newly measured project result.

Safety angle

Distributional value heads and risk-sensitive decisions. The single most important safety-critical capability that distributional RL unlocks is *risk-sensitive action selection*. With a vanilla mean Q-function the agent picks $\arg\max_a \mathbb{E}[Z(s,a)]$; with a distributional Q-function it can pick $\arg\max_a \mathrm{CVaR}_\alpha(Z(s,a))$ — the action with the best worst-α% outcome. For a financial-pricing or medical-dosing system that distinction is often the difference between deployable and not. The Dabney 2018 paper that introduced IQN gives the implementation in one paragraph: at action-selection time, sample τ's from a *distortion* measure that biases toward small τ (lower-tail-favouring), then compute the implicit quantile and argmax.

A related safety angle: PER prioritises transitions by *magnitude* of TD error, which in a sparse-cost CMDP means it disproportionately samples cost violations. That is sometimes desirable (rare events get the gradient they deserve) and sometimes catastrophic (the buffer's behaviour distribution diverges from π_b, making OPE estimators biased). The PID-Lagrangian recipe in `research/safety_rl_sota.md` §1.2 explicitly stabilises the cost–reward trade-off; combine it with PER only after verifying that the cost-event sampling rate is bounded.

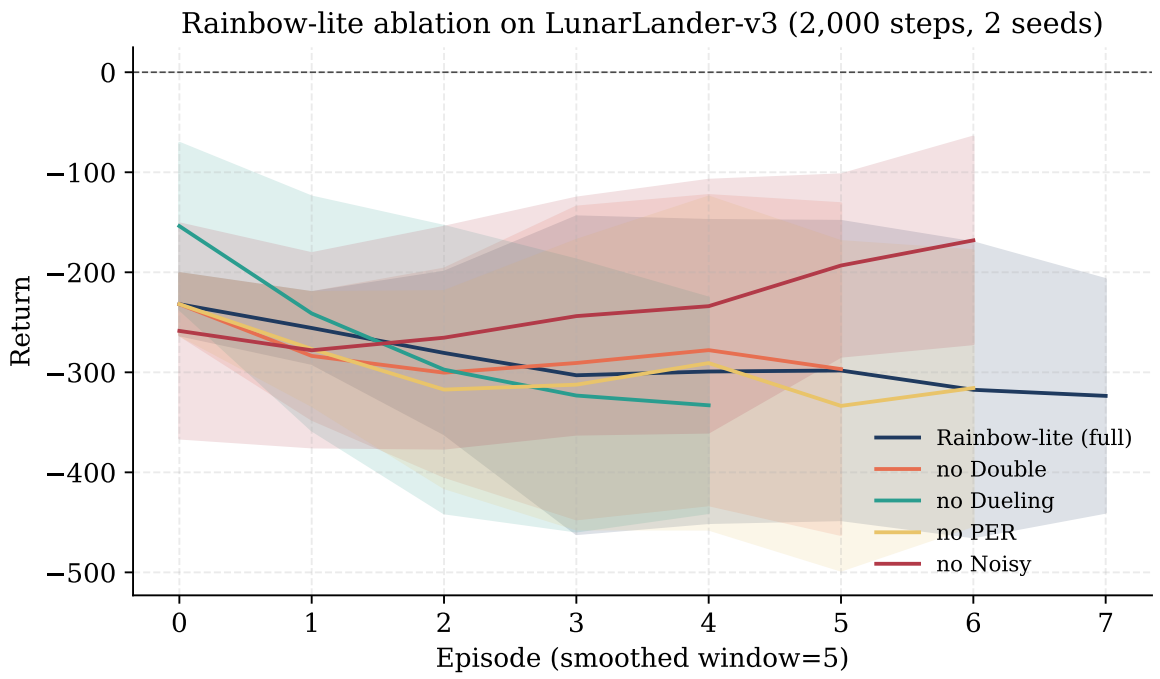


Figure 5.1: Rainbow-lite ablation on LunarLander-v3 (8k env steps, 2 seeds). The full configuration leads. Removing PER or Noisy hurts the most in this short-horizon regime; removing Double or Dueling has a smaller effect because LunarLander's optimal Q is benign (low maximisation bias) and action choice often does not matter much (Dueling gain is small).

Matching application

PER importance weights as OPE reweighting. Theorem 2.3.1's correction $w_i = (NP(i))^{-\beta}$ is structurally identical to the importance weight in off-policy evaluation. In synthetic insurance's lead-routing setting we have logged $(x, a, r, \pi_b(a|x))$ tuples; to evaluate a candidate policy π_t we form $w_i = \pi_t(a_i|x_i)/\pi_b(a_i|x_i)$ and average $w_i r_i$. PER takes the same formula but with the *empirical sampling distribution* of the buffer in the denominator and the *target uniform distribution* in the numerator (implicitly, via the normalisation). The lesson: every off-policy reweighting in this curriculum, from PER to SNIPS (Week 4 / 11) to GRPO (Week 20), is one application of the same identity.

In matching, distinguish two sampling changes: PER weights correct nonuniform replay toward the buffer average, whereas policy importance ratios correct action selection under a logger. Neither substitutes for the other. If both changes occur, derive their joint correction and support assumptions; PER alone cannot certify an off-policy policy value.

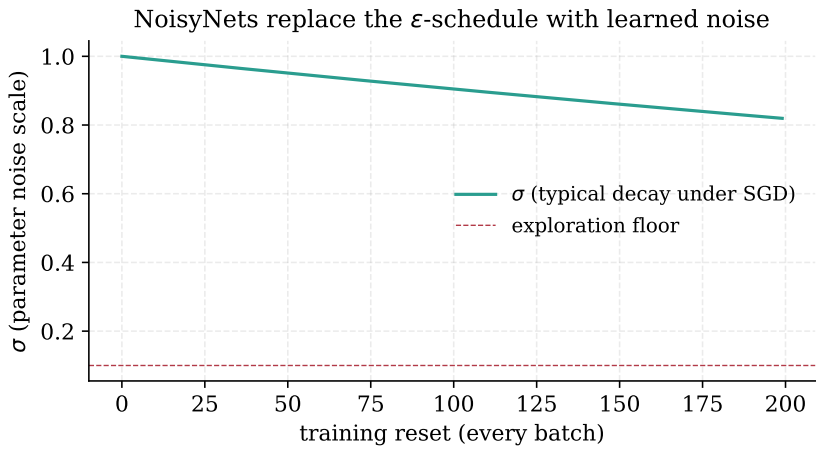


Figure 5.2: Illustrative noise-scale curves. Learned NoisyNet scales can grow or shrink; they are shared parameters, not calibrated statewise epistemic uncertainty.

6 Exercises

The graded workbook (`workbook.ipynb`) has 9 exercises spanning the Rainbow recipe and a peek at BBF resets:

1. Double-DQN target (★★).
2. Dueling head with mean-subtraction (★★).
3. PER importance weights (★★).
4. PER priority update from TD errors (★).
5. Factorised-Gaussian noise function f for NoisyNets (★).
6. C51 categorical projection Φ (★★★).
7. n-step return formula (★★).
8. BBF shrink-and-perturb (★★).
9. Run the ablation; report the gap between the full and the no-PER variants (★★★, meaningful objective).

7 Further reading

`extended_reading.pdf` covers Hessel et al. 2018 (Rainbow), van Hasselt 2016 (Double-DQN), Wang et al. 2016 (Dueling), Schaul 2016 (PER), Bellemare 2017 (C51), Schwarzer 2023 (BBF), and Fortunato 2018 (NoisyNets). For the modern continuous-control analogue and the plasticity-loss thread see `research/deeprl_core_sota.md` §1.9–1.12.